

РОЗДІЛ 8.

КОМП'ЮТЕРНА ТА ПРОГРАМНА ІНЖЕНЕРІЯ

Murin Oleksandr

Higher education applicant of the Faculty of Computer Science
Kharkiv National University of Radio Electronics, Ukraine

Scientific supervisor: Anastasia Chuprina

Associate Professor of the Department of Software Engineering,
Candidate of Technical Sciences, Associate Professor
Kharkiv National University of Radio Electronics, Ukraine

STRUCTURAL NAVIGATION COMPLEXITY AS A CRITERION FOR FRONTEND ARCHITECTURE EVALUATION

SUMMARY. Modern frontend applications grow in size and complexity as functionality expands, making project structure an important factor in understanding and maintaining the system. This study compares three frontend architectural approaches in a Next.js application: monolithic architecture, modular architecture, and Feature-Sliced Design. Each approach was implemented within the same application and evaluated across the same set of user scenarios. To compare the architectures, a navigation-based model was introduced, representing the project structure as a tree. Based on this model, several metrics were calculated, including depth, coverage, precision, visited nodes, and navigation cost. The results show that monolithic architecture provides the lowest depth but exposes many irrelevant elements, resulting in high coverage and low precision. Modular architecture achieved the best overall result in the conducted experiment, with the lowest number of visited nodes, the best average precision, and the lowest navigation cost. Feature-Sliced Design provides the lowest coverage, indicating strong separation of unrelated functionality, but introduces additional structural overhead. The findings suggest that no architecture is universally optimal: monolithic architecture is suitable for smaller projects, modular architecture is the most effective option for the evaluated project scale, and Feature-Sliced Design becomes more relevant when stronger separation and long-term scalability are required.

KEYWORDS: *frontend architecture, Next.js, software architecture, navigation complexity, Feature-Sliced Design, modular architecture.*

Introduction. Modern frontend applications are becoming increasingly complex as their functionality and codebases grow. As a result, the organization of the codebase plays an important role in determining how easily new features can be implemented and existing logic can be understood.

One of the key factors influencing these properties is the architectural approach used to structure a frontend application. Architectural decisions define

how responsibilities are distributed, how components interact, and how changes propagate through the system. In practice, poorly structured architecture often leads to increased coupling, reduced maintainability, and higher cognitive complexity. This problem is particularly relevant in the context of modern frameworks such as Next.js. While Next.js provides a powerful set of features for building applications, it does not impose a strict architectural structure. As a result, developers are free to choose how to organize the codebase, which leads to the coexistence of different architectural approaches, such as monolithic organization, modular architecture, and Feature-Sliced Design.

Studies on program comprehension show that developers understand software systems through an iterative process of information seeking and hypothesis validation. LaToza and Myers [1] demonstrate that developers frequently ask reachability questions, which require navigating across different parts of the codebase. Furthermore, program comprehension has been described as a fact-finding process, in which developers locate and connect relevant pieces of information within the system [2]. These observations suggest that the structure of a project may influence the effort required to locate relevant code artifacts.

Despite the widespread use of different architectural approaches, their comparison is typically based on qualitative analysis rather than measurable characteristics. In particular, limited attention is given to how architectural structure affects the process of navigating the codebase and locating the files required to understand or modify a specific feature.

To address this limitation, the present study proposes a quantitative approach to evaluating the structural complexity of frontend applications. An experimental study is conducted in which a functionally identical application is implemented using different architectural approaches and analyzed across multiple development scenarios. The results provide insights into how architectural decisions influence the structure of the project and the effort required to work with its codebase.

Related Works. Software architecture and code quality evaluation have been widely studied using established software metrics. Chidamber and Kemerer proposed a metrics suite for object-oriented design that includes measures related to coupling, cohesion, inheritance, and class complexity [3]. McCabe introduced cyclomatic complexity as a measure of the number of independent paths through program logic [4]. Oman and Hagemeister proposed maintainability-related metrics and discussed how they can be combined into a maintainability index [5]. These approaches provide quantitative indicators of internal code complexity and maintainability, focusing primarily on relationships between program elements.

In frontend development, architectural organization is commonly discussed through practical approaches such as monolithic structure, modular architecture,

РОЗДІЛ 8.

КОМП'ЮТЕРНА ТА ПРОГРАМНА ІНЖЕНЕРІЯ

and Feature-Sliced Design. The official Feature-Sliced Design documentation describes FSD as a methodology for organizing scalable frontend applications using a layered and feature-oriented structure [6]. These approaches are typically evaluated in terms of maintainability, scalability, and separation of concerns, with emphasis on design principles and developer experience.

The importance of software architecture for system quality is emphasized in general architecture literature. Robert C. Martin argues that architecture should minimize the effort required to build and maintain a system [7]. Bass, Clements, and Kazman describe software architecture as a key factor influencing system quality attributes such as modifiability and maintainability [8]. These works provide a conceptual foundation for understanding the role of architecture in software systems.

Studies on program comprehension provide insights into how developers explore and understand large codebases. LaToza and Myers show that developers frequently ask reachability questions when analyzing software, which involves locating relevant elements across different parts of the system [1]. LaToza, Garlan, Herbsleb, and Myers further describe program comprehension as a fact-finding process in which developers iteratively locate, evaluate, and connect pieces of information within the codebase [2].

Despite the substantial body of work on code quality metrics, architectural design, and program comprehension, the problem of navigating project structure in frontend applications remains less explicitly addressed. In particular, there is limited quantitative analysis of how different architectural approaches influence the effort required to locate scenario-related files within the project file tree. The present study addresses this gap by modeling the project structure as a navigational space and evaluating different architectural approaches using structural metrics such as depth, coverage, precision, visited nodes, and a heuristic navigation score.

Methodology. The objective of this study is to quantitatively evaluate how different frontend architectural approaches influence the structural complexity of navigating a project codebase. The evaluation focuses on the effort required to locate and understand files related to specific user scenarios.

To ensure comparability, three architectural approaches were implemented within the same application: monolithic architecture, modular architecture, and Feature-Sliced Design. Each implementation provides identical functionality and is extended using the same set of user scenarios. This controlled setup allows the analysis to isolate the effect of architectural structure on navigation complexity. The project structure of each implementation is modeled as a hierarchical tree, where directories represent intermediate nodes and files represent leaf nodes. This representation allows the project to be interpreted as a navigational space, in which

developers traverse directories and inspect files in order to locate relevant elements.

Based on observations from program comprehension studies, navigation is modeled as an iterative process in which a developer explores the structure step by step, expanding directories and inspecting their contents until the required files are found. For each scenario, a set of required files is defined, representing the minimal subset of files necessary to understand or modify the corresponding feature. During navigation, the developer is assumed to observe not only the required files but also intermediate and irrelevant elements encountered along the traversal path. These elements are classified into three categories: required nodes (files directly related to the scenario), access nodes (nodes that lie on the path to required files), and visible nodes (all nodes observed during traversal).

To evaluate the structural properties of navigation, several metrics are defined. Depth measures the number of hierarchical levels required to reach relevant files. Coverage represents the proportion of visible nodes relative to the total number of nodes at a given level, reflecting how much of the structure must be inspected. Precision measures the proportion of required nodes among the visible nodes, indicating how much of the observed information is relevant.

The navigation cost metric is defined as a heuristic measure of the structural effort required to locate all required files within the project tree for a given scenario. The visible scope is constructed by tracing, for each required file, the access path from the project root to that file. At each directory along this path, all immediate children are added to the visible scope, reflecting the assumption that a developer who opens a directory observes all of its direct contents. Each visible node is then assigned a base cost according to its type: a file contributes 1.0 unit and a directory contributes 0.5 units. This distinction reflects the observation that files typically require more focused inspection, as they contain the actual implementation, whereas directories serve primarily as navigational containers.

In cases where a directory contains more than one required file as direct children, an additional cost is introduced to reflect the repeated inspection of that directory's contents. For such a directory d , the extra cost is computed as equation (1):

$$ExtraCost(d) = \left(\sum_{c \in C(d)} w(c) \right) \times 0,5 \times (|R(d)| - 1), \quad (1)$$

where

$C(d)$ – denotes the set of visible immediate children of d ;

$w(c)$ – is the weight of node c (0.5 for a directory, 1.0 for a file);

$R(d)$ – is the set of required files located directly in d .

РОЗДІЛ 8.

КОМП'ЮТЕРНА ТА ПРОГРАМНА ІНЖЕНЕРІЯ

The factor of 0.5 reflects the assumption that revisiting a previously inspected directory is less costly than the initial traversal, as part of its contents is already familiar to the developer. The extra cost is distributed evenly across all visible children of the directory.

The Navigation Score for a scenario is computed as the sum of base costs and extra costs across all visible nodes, as shown in equation (2):

$$NS = \sum_{n \in V} w(n) + \sum_{d \in D, |R(d)| > 1} ExtraCost(d), \quad (2)$$

A lower Navigation Score indicates that fewer structural elements are involved in locating the required files. The values computed using equations (1) and (2) for each scenario and architecture are presented in Table 1.

The evaluation is performed across multiple user scenarios, each corresponding to a specific feature of the application, such as viewing posts, authentication, creating content, and interacting with posts. For each scenario and each architectural approach, the required files are identified and the corresponding navigation process is simulated. All metrics are then computed based on the resulting traversal.

This methodology enables a structured comparison of architectural approaches by linking project organization to measurable navigation characteristics, providing insight into how different structures affect the effort required to work with the codebase.

Experiments. The experimental application was implemented as a content platform built with Next.js App Router, React, TypeScript, Material UI, and a REST API backend. The application supports ten user scenarios: viewing the post feed, viewing the authenticated user's post feed, registration, authorization, creating a post, updating a post, deleting a post, adding a comment, toggling a post like, and searching, filtering, and sorting the feed. The same set of scenarios and identical functionality was implemented across all three architectural variants, ensuring that differences in metric values reflect structural organization rather than functional scope.

For each scenario, the set of required files was identified by manually inspecting each architectural variant and selecting the minimal subset of files necessary to understand or modify the corresponding feature. For example, the update post scenario in the monolithic variant includes the edit page, the post form component, the edit button component, the post card component, and the relevant API action files.

To support the analysis, a dedicated Navigation Complexity Visualizer was developed as a separate Next.js application. The tool constructs a tree

representation of each project from predefined structural data and, for a selected scenario, highlights required nodes, access path nodes, and visible nodes using distinct visual markers. The visualizer was used to validate the identified required file sets and to confirm that the computed navigation scope matched the actual project structure. All metric values reported in the results were calculated by the visualizer based on the navigation model defined in the methodology.

Results. The results reveal consistent differences in navigation behavior across all three architectural approaches. Monolithic architecture produces the shallowest but noisiest structure, modular architecture demonstrates the most efficient overall navigation behavior for the evaluated project scale, and Feature-Sliced Design achieves the strongest separation of unrelated functionality at the cost of additional structural overhead. Table 1 presents detailed measurements for each development scenario, and Table 2 presents aggregated metrics summarizing the overall structural characteristics of each approach.

Table 1

Scenario-level navigation complexity comparison

Feature	Architecture	FTUF	Visited nodes	Depth	Coverage	Precision	Navigation Score
Post feed	Monolith	4	43	3	82,70%	23,30%	37,5
	Modular	4	38	5	56,70%	34,20%	29,5
	FSD	6	40	4	33,60%	40,00%	32,5
User's post feed	Monolith	3	43	4	75,40%	20,90%	37
	Modular	3	35	5	52,20%	31,40%	27,5
	FSD	5	40	4	33,60%	37,50%	32
Registration	Monolith	4	42	4	73,70%	26,20%	36
	Modular	4	28	4	43,80%	46,40%	21
	FSD	4	37	4	31,10%	35,10%	26
Authorization	Monolith	4	42	4	73,70%	26,20%	36
	Modular	4	28	4	43,80%	46,40%	21
	FSD	4	37	4	31,10%	35,10%	26
Create post	Monolith	4	40	4	70,20%	22,50%	37,25
	Modular	4	34	5	50,70%	35,30%	27
	FSD	4	41	4	34,50%	31,70%	30

РОЗДІЛ 8.

КОМП'ЮТЕРНА ТА ПРОГРАМНА ІНЖЕНЕРІЯ

Continuation of table 1

Feature	Architecture	FTUF	Visited nodes	Depth	Coverage	Precision	Navigation Score
Delete post	Monolith	3	28	2	65,10%	17,90%	31,5
	Modular	3	21	4	32,80%	33,30%	19
	FSD	4	29	4	24,40%	37,90%	21,5
Update post	Monolith	6	43	6	72,90%	30,20%	53,25
	Modular	6	37	6	54,40%	43,20%	35
	FSD	7	51	6	42,10%	43,10%	37,5
Add comment	Monolith	5	28	2	65,10%	25,00%	45,5
	Modular	5	26	4	40,60%	42,30%	25,5
	FSD	5	29	4	24,40%	41,40%	23,5
Toggle like	Monolith	4	28	2	65,10%	21,40%	38,5
	Modular	4	21	4	32,80%	38,10%	22
	FSD	4	28	4	23,50%	39,30%	20,5
Search/ filter/ sort	Monolith	4	43	3	82,70%	23,30%	37,5
	Modular	4	38	5	56,70%	34,20%	29,5
	FSD	4	41	4	34,50%	31,70%	28

FTUF (files to understand feature). Monolithic and modular architectures show identical average FTUF values, while FSD requires a slightly larger number of files in several scenarios. This can be explained by the stronger decomposition of functionality in FSD.

Visited nodes. Modular architecture achieves the best result in all evaluated scenarios, indicating that its structure limits the navigation scope most effectively. FSD shows the highest number of visited nodes in four scenarios, which can be attributed to the larger number of directories and intermediate structural elements. Monolithic architecture shows the highest number of visited nodes in five scenarios, mainly due to the concentration of unrelated files within broad shared directories.

Depth. Monolithic architecture achieves the best depth values in five out of ten scenarios, which reflects its flatter structure. Modular architecture shows the highest depth in four scenarios, indicating that its organization introduces additional navigation levels in exchange for better localization of functionality.

Coverage. FSD achieves the best coverage values in all ten scenarios, meaning that a smaller part of the overall project structure becomes visible while locating

scenario-related files. This result indicates stronger separation of unrelated functionality. At the same time, the low coverage values of FSD should be interpreted together with the larger size of its project tree, since the architecture introduces more structural elements overall. Monolithic architecture shows the worst coverage values in all scenarios, which reflects the fact that its flatter and broader structure exposes a larger share of unrelated elements during navigation.

Precision. Modular architecture achieves the best precision in six scenarios, while FSD achieves the best precision in four scenarios. This indicates that both approaches improve the relevance of inspected elements compared to the monolithic structure. Monolithic architecture shows the worst precision values, which is consistent with its high coverage and the presence of many unrelated elements in the visible navigation scope.

Navigation Score. Modular architecture achieves the best result in seven scenarios, indicating the most efficient overall navigation behavior. This can be explained by its domain-oriented structure, where related files are grouped within coherent modules. FSD achieves the best result in three scenarios, benefiting from strong separation of functionality but also introducing additional structural overhead. Monolithic architecture shows the worst Navigation Score in all scenarios, mainly because many scenario-related and unrelated files are concentrated in the same broad directories, increasing the cost of inspection.

Table 2

Average navigation complexity comparison

Architecture	FTUF	Visited nodes	Depth	Coverage	Precision	Navigation Score
Monolith	4,10	38,00	3,40	72,66%	23,69%	39,00
Modular	4,10	30,60	4,60	46,45%	38,48%	25,70
FSD	4,70	37,30	4,20	31,28%	37,28%	27,75

The average values confirm that monolithic architecture provides the shallowest structure, with an average depth of 3.40. This means that scenario-related files can generally be reached through shorter navigation paths. However, this advantage is accompanied by the highest average coverage value of 72.66% and the lowest average precision of 23.69%. Therefore, monolithic structure reduces path length but exposes a significantly larger portion of irrelevant project elements during navigation.

Modular architecture demonstrates the strongest overall balance between the evaluated metrics. It has the same average FTUF value as monolithic architecture (4.10) but requires fewer visited nodes on average (30.60 compared to 38.00). This

РОЗДІЛ 8.

КОМП'ЮТЕРНА ТА ПРОГРАМНА ІНЖЕНЕРІЯ

means that, although the number of files required to understand a feature is similar, the modular structure reduces the amount of surrounding project structure that must be inspected. It also achieves the best average precision (38.48%) and the lowest average navigation score (25.70), which indicates the most efficient navigation behavior in the conducted experiment.

Feature-Sliced Design achieves the lowest average coverage value (31.28%), showing that unrelated functionality is more effectively separated from the navigation scope. However, this result should be interpreted together with FTUF and visited nodes. FSD has the highest average FTUF value (4.70) and an average of 37.30 visited nodes, which is close to the monolithic value. This indicates that FSD hides unrelated parts of the system well in relative terms but introduces additional structural elements that increase the amount of navigation required in absolute terms.

These results show that the architectures differ not only in numerical metric values, but also in the shape of the project structure they produce. Monolithic architecture can be interpreted as a shallow tree with a limited number of broad branches and a high concentration of files inside shared directories. This explains its low depth, but also its high coverage and low precision. Modular architecture forms a more balanced tree, where files are grouped around domain-oriented modules. This reduces the number of irrelevant elements visible during navigation without introducing excessive fragmentation. FSD creates a more segmented tree with a larger number of intermediate directories and clearer separation between layers, features, entities, and widgets. This improves localization but increases structural overhead.

The scale of the experimental application should also be considered when interpreting the results. The evaluated project is relatively small compared to industrial frontend systems. Therefore, the observed differences should not be interpreted as absolute values applicable to all projects, but rather as structural tendencies.

Conclusions. Conducted experiment shows that the same functionality, implemented in different architectural variants, produced different navigation patterns, which confirms that project organization can meaningfully affect the process of locating scenario-related code.

Monolithic architecture proved to be the simplest in terms of path length. Its flat structure makes files easy to reach, which can be convenient at the early stages of development. However, this simplicity comes with a significant drawback: unrelated functionality is concentrated in broad shared directories. As a result, the developer encounters more irrelevant elements while navigating the project. Therefore, monolithic approach can be considered acceptable for small applications, but it becomes less suitable as the amount of functionality increases.

Modular architecture demonstrated the most balanced behavior in the experiment. Grouping code around domain-oriented modules reduced the amount of surrounding structure involved in navigation while avoiding excessive fragmentation. This makes modular approach a practical compromise for applications of small and medium complexity, where maintainability is important but the overhead of a stricter architecture may not yet be justified.

Feature-Sliced Design showed the strongest separation of unrelated functionality. Its structure makes it easier to isolate a specific functional area from the rest of the project. At the same time, this approach introduces more intermediate directories and requires a clearer understanding of architectural rules. For this reason, FSD may appear excessive in a small project, but its advantages are expected to become more significant as the number of features, entities, and independent development areas increases.

Overall, the choice of architecture should be guided by the expected growth trajectory of the application rather than its initial size alone. As a project scales, the structural tendencies observed in this study are expected to intensify: monolithic architecture will accumulate more files within broad shared directories, further reducing precision; modular architecture will distribute growth across domain modules, preserving navigational efficiency but potentially increasing the width of individual modules; and Feature-Sliced Design will grow through additional feature and entity branches, maintaining separation of unrelated functionality at the cost of a larger number of structural elements. These tendencies suggest that navigation-based evaluation could serve as a useful complement to existing architecture assessment approaches, and that future work could extend this model to larger projects or validate it through observational studies with real developers.

References:

- [1] LaToza T. D., Myers B. A. Developers ask reachability questions. ICSE. 2010. P. 185–194. URL: <https://dl.acm.org/doi/10.1145/1806799.1806829> (date of access: 27.04.2026).
- [2] LaToza T. D., Garlan D., Herbsleb J. D., Myers B. A. Program comprehension as fact finding. ESEC/FSE. 2007. P. 361–370. URL: <https://dl.acm.org/doi/10.1145/1287624.1287675> (date of access: 27.04.2026).
- [3] Chidamber S. R., Kemerer C. F. A metrics suite for object oriented design. IEEE Transactions on Software Engineering. 1994. Vol. 20, no. 6. P. 476–493. URL: <https://dl.acm.org/doi/10.1109/32.295895> (date of access: 27.04.2026).
- [4] McCabe T. J. A complexity measure. IEEE Transactions on Software Engineering. 1976. Vol. SE-2, no. 4. P. 308–320. URL: <https://dl.acm.org/doi/10.1109/TSE.1976.233837> (date of access: 27.04.2026).
- [5] Oman P., Hagemeister J. Metrics for assessing a software system's maintainability. Conference on Software Maintenance. 1992. P. 337–344. URL: <https://www.computer.org/csdl/proceedings-article/icsm/1992/00242525/12OmNyrqzy2> (date of access: 27.04.2026).
- [6] Feature-Sliced Design. Documentation. URL: <https://feature-sliced.design/> (date of access: 27.04.2026).
- [7] Martin R. C. Clean Architecture: A Craftsman's Guide to Software Structure and Design. Boston: Prentice Hall, 2017. 432 p.
- [8] Bass L., Clements P., Kazman R. Software Architecture in Practice. 3rd ed. Boston: Addison-Wesley, 2012. 624 p.